МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №2
з дисципліни «Професійний Python»

Тема: «Структури даних та функціональна декомпозиція»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Сформувати практичні навички використання базових і розширених структур даних Python (list, tuple, set, dict, модуль collections), навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чіткою відповідальністю, використовуючи *args, **kwargs, lambda, closures і decorators.

1. Постановка задачі

Варіант 1. Аналіз успішності студентів (узгоджено з ЛР1 «Система обліку студентів»). Вхідні дані – набір студентів з полями: ім'я, прізвище, група, середній бал. Потрібно створити модуль обробки структурованих даних, який реалізує операції з таблиці 1, використовує всі основні структури даних, comprehensions, функції вищого порядку, closure і decorator, а також містить аналіз складності та benchmark пошуку.

Таблиця 1 – Вимоги варіанта 1 та їх реалізація (результат на демонстраційних даних)

Вимога	**Функція**	**Результат**
пошук студента за ID	`find_student`	3 → Ірина Бондар
пошук за прізвищем (без урахування регістру)	`find_by_last_name`	«Мельник» → Олег Мельник, Максим Мельник
групування за групою	`group_students`	PI-21 – 4, PI-22 – 3, PI-23 – 3
середній бал усіх студентів	`calculate_average`	80.05
найкращий студент	`find_best_student`	Ірина Бондар, 95.20
фільтрація за мін. балом (85)	`filter_by_min_grade`	ID 1, 3, 4, 9
сортування за середнім балом	`sort_by_grade`	ID 3, 9, 1 …
dict-index за ID студента	`create_student_index`	10 ключів

2. Опис структури даних

Один студент – це словник (dict) з п'ятьма полями (таблиця 2). Такий формат відповідає JSON-like даним, легко серіалізується і дозволяє звертатися до полів за ключем. Набір студентів зберігається у списку (list) `students` – це впорядкована колекція, яку можна фільтрувати, сортувати й групувати. Демонстраційні дані задано таблицею кортежів `STUDENT_ROWS`, з якої списком comprehension через `create_student` будуються записи; для великих наборів функція `generate_students(count, seed)` генерує відтворювані випадкові дані. Демонстраційний набір містить 10 студентів у 3 групах, 9 унікальних прізвищ (прізвище «Мельник» трапляється двічі, тому пошук за прізвищем повертає список).

Таблиця 2 – Поля запису студента

Ключ	Тип	Приклад	Зміст
`id`	int	3	ідентифікатор студента (унікальний ключ індексу)
`first_name`	str	"Ірина"	ім'я
`last_name`	str	"Бондар"	прізвище
`group`	str	"PI-22"	група (не порожня)
`average_grade`	float	95.2	середній бал, 0–100

3. Вибір структур даних

Таблиця 3 – Використані структури даних і причини вибору

Структура	Де використано	Чому саме вона
list	`students`, результати фільтрації та сортування	впорядкований змінюваний набір; підтримує сортування і зріз
tuple	`STUDENT_ROWS`, `GRADE_LEVELS`, `GROUPS`, ключ сортування у `sort_items`, `GradeStats` (NamedTuple)	незмінні групи значень; порівнюються лексикографічно; hashable
set / frozenset	`get_unique_values`, фільтр груп у closure, `options.keys() - SUMMARY_DEFAULTS.keys()`	унікальність і перевірка `in` за O(1); операції над множинами
dict	запис студента, dict-індекс за ID, `LEVEL_LABELS`, підсумок	доступ за ключем O(1); природне представлення запису
defaultdict(list)	`group_students`, `group_nested`	групування без перевірки, чи існує ключ
Counter	`count_students_by_group`, `operation_counts`	підрахунок частот, `most_common()`
deque(maxlen=5)	`operation_history`	журнал останніх операцій: append O(1), старі записи відкидаються

Приклад використання collections – групування з defaultdict і підрахунок з Counter:

```
@track_operation("групування за групою")
def group_students(students: list[Student]) -> dict[str, list[Student]]:
    groups: defaultdict[str, list[Student]] = defaultdict(list)
    for student in students:
        groups[student["group"]].append(student)
    return dict(groups)

def count_students_by_group(students: list[Student]) -> Counter[str]:
    return Counter(student["group"] for student in students)
```

4. Структура проєкту

Проєкт має src-layout, як у ЛР1, і встановлюється командою `python -m pip install -e .`; точки входу – `python -m data_processor.main` і `python -m data_processor.benchmark` (або console commands `student-report`, `student-benchmark`).

```
Lab_02/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── data_processor/
│       ├── __init__.py
│       ├── analytics.py
│       ├── benchmark.py
│       ├── data.py
│       ├── decorators.py
│       ├── main.py
│       └── processors.py
├── tests/
│   ├── test_analytics.py
│   └── test_processors.py
├── results/        summary.json, benchmark.csv
└── report/         build_report.py (генератор цього звіту)
```

Таблиця 4 – Призначення модулів

Модуль	**Відповідальність**
`data.py`	вхідні дані, константи, `create_student`, `generate_students`
`decorators.py`	`measure_time`, параметризований `track_operation`, журнал deque і Counter
`processors.py`	індекс, пошук, фільтри (closures), групування, сортування, pipeline
`analytics.py`	середнє, найкращий студент, статистика, підсумок, експорт у JSON
`benchmark.py`	експериментальна частина: list vs dict/set
`main.py`	точка входу: демонстрація всіх операцій

5. Основні функції

Програму розбито на 44 функції верхнього рівня у 7 модулях; кожна виконує одну логічну задачу (таблиця 5).

Таблиця 5 – Основні функції модуля

Функція	**Призначення**	**Прийом**
`create_student_index(students)`	dict-індекс за ID	dict comprehension
`find_student(index, student_id)`	пошук за ID	dict.get, декоратор
`find_by_last_name(students, last_name)`	пошук за прізвищем	list comprehension, casefold
`filter_items(items, predicate)`	універсальний фільтр	функція вищого порядку
`create_grade_filter(minimum)`	фільтр за мін. балом	closure
`create_group_filter(*groups)`	фільтр за групами	closure, *args
`group_students`, `group_nested`	групування, вкладене групування	defaultdict, рекурсія
`sort_by_grade`, `sort_items`	сортування за балом / за кількома полями	lambda, tuple-ключ
`calculate_average`	середній бал	generator expression, декоратор
`calculate_stats(*grades)`	статистика значень	*args, NamedTuple
`build_summary(students, **options)`	підсумковий звіт	**kwargs

Функція	Призначення	Прийом
`run_pipeline(items, *steps)`	послідовна обробка	reduce + lambda

6. Приклади ключових конструкцій

6.1. Comprehensions і generator expression

Set comprehension збирає унікальні значення поля, dict comprehension будує індекс, list comprehension фільтрує, а generator expression рахує суму без проміжного списку:

```
def get_unique_values(students: list[Student], field: str) -> set[Any]:
    return {student[field] for student in students}

def create_student_index(students: list[Student]) -> dict[int, Student]:
    index = {student["id"]: student for student in students}
    if len(index) != len(students):
        raise ValueError("Ідентифікатори студентів повторюються")
    return index

def filter_items(items: list[Student], predicate: Predicate) -> list[Student]:
    return [item for item in items if predicate(item)]

@measure_time
def calculate_average(students: list[Student]) -> float:
    if not students:
        return 0.0
    return round(sum(student["average_grade"] for student in students) / len(students), 2)
```

6.2. *args

Функція `calculate_stats` приймає довільну кількість значень; при виклику генератор розпаковується оператором *: `calculate_stats(*(s["average_grade"] for s in students))`.

```
def calculate_stats(*grades: float) -> GradeStats:
    if not grades:
        return GradeStats(0, 0.0, 0.0, 0.0)
    return GradeStats(
        size=len(grades),
        average=round(sum(grades) / len(grades), 2),
        minimum=min(grades),
        maximum=max(grades),
    )
```

6.3. **kwargs

`create_student(**fields)` формує запис з іменованих аргументів і перевіряє його: усі поля на місці, група не порожня, середній бал у межах 0–100. `build_summary(students, **options)` приймає налаштування `top` і `min_grade`; невідомі ключі знаходяться різницею множин ключів і дають `TypeError`.

```
def create_student(**fields: Any) -> Student:
    missing = [name for name in REQUIRED_FIELDS if name not in fields]
    if missing:
        raise ValueError(f"Відсутні поля: {', '.join(missing)}")
    if not str(fields["group"]).strip():
        raise ValueError("Група не може бути порожньою")
    if not 0 <= fields["average_grade"] <= 100:
        raise ValueError("Середній бал має бути в межах 0-100")
    return dict(fields)
```

6.4. lambda

lambda використано як короткий ключ сортування та пошуку максимуму. У `sort_items` ключ – це tuple значень кількох полів, тому студенти впорядковуються спершу за групою, потім за середнім балом.

```
@track_operation("сортування за балом")
def sort_by_grade(students: list[Student], reverse: bool = True) -> list[Student]:
    return sorted(students, key=lambda student: student["average_grade"], reverse=reverse)

def sort_items(items: list[Student], *fields: str, reverse: bool = False) ->
list[Student]:
    if not fields:
        raise ValueError("Потрібно вказати хоча б одне поле для сортування")
    return sorted(
        items,
        key=lambda item: tuple(item[field] for field in fields),
        reverse=reverse,
    )
```

6.5. Closure

`create_grade_filter` повертає внутрішню функцію `predicate`, яка пам'ятає `minimum` після завершення зовнішньої функції (значення зберігається в `predicate.__closure__`, що перевіряє тест). Так само `create_group_filter(*groups)` замикає frozenset допустимих груп.

```
def create_grade_filter(minimum: float) -> Predicate:
    def predicate(student: Student) -> bool:
        return student["average_grade"] >= minimum

    return predicate

def create_group_filter(*groups: str) -> Predicate:
    allowed = frozenset(groups)

    def predicate(student: Student) -> bool:
        return student["group"] in allowed

    return predicate
```

6.6. Decorators

`measure_time` виводить час виконання функції, не змінюючи її коду (застосовано до `calculate_average`). `track_operation(label)` – параметризований декоратор: зовнішня функція приймає параметр, повертає декоратор, а wrapper після кожного виклику записує операцію в deque та Counter. `functools.wraps` зберігає ім'я та docstring обгорнутої функції.

```
def measure_time(func: Callable[P, R]) -> Callable[P, R]:
    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"[час] {func.__name__}: {elapsed:.6f} с")
        return result

    return wrapper

def track_operation(
    label: str,
    *,
```

```
    history: deque[str] | None = None,
    counts: Counter[str] | None = None,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    target_history = operation_history if history is None else history
    target_counts = operation_counts if counts is None else counts

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            result = func(*args, **kwargs)
            target_history.append(label)
            target_counts[label] += 1
            return result

        return wrapper

    return decorator
```

7. Результат роботи програми

Вивід команди `python -m data_processor.main`:

```
ЛР2, варіант 1: аналіз успішності студентів

Усі студенти (10)
------------------------------------------------------------
  1 Анна Коваль              PI-21     91.50
  2 Олег Мельник             PI-21     84.00
  3 Ірина Бондар             PI-22     95.20
  4 Андрій Шевченко          PI-21     88.70
  5 Марія Бойко              PI-22     79.30
  6 Тарас Гриценко           PI-23     72.40
  7 Софія Гнатюк             PI-22     66.80
  8 Дмитро Ткаченко          PI-23     58.90
  9 Катерина Лисенко         PI-23     93.10
 10 Максим Мельник           PI-21     70.60

Унікальні групи (set): ['PI-21', 'PI-22', 'PI-23']
Чи є група PI-24: False
Унікальних прізвищ: 9 на 10 студентів

Пошук за ID (dict-індекс):
  3: Ірина Бондар, PI-22, 95.2
  99: не знайдено
  Лінійний пошук у list дає той самий об'єкт: True
Пошук за прізвищем 'Мельник': ['Олег Мельник', 'Максим Мельник']
Пошук за прізвищем 'бойко': ['Марія Бойко']

Групування за групою (defaultdict):
  PI-21: Коваль, Мельник, Шевченко, Мельник
  PI-22: Бондар, Бойко, Гнатюк
  PI-23: Гриценко, Ткаченко, Лисенко
Кількість студентів (Counter.most_common): PI-21=4, PI-22=3, PI-23=3
Вкладене групування група -> рівень успішності:
  PI-21: відмінно=1, добре=2, задовільно=1
  PI-22: відмінно=1, добре=1, задовільно=1
  PI-23: задовільно=1, незадовільно=1, відмінно=1
[час] calculate_average: 0.000005 с

Середній бал усіх студентів: 80.05
Найкращий студент: Ірина Бондар (95.20)

Статистика за групами (dict comprehension + NamedTuple):
  група    к-сть  середній    мін.   макс.
  PI-21        4     83.70    70.6    91.5
  PI-22        3     80.43    66.8    95.2
  PI-23        3     74.80    58.9    93.1

Фільтр average_grade >= 85 (closure): [1, 3, 4, 9]
Фільтр average_grade < 70 (lambda): [7, 8]
Студенти групи PI-22: [3, 5, 7]
```

```
Рейтинг за середнім балом (спадання)
------------------------------------------------------------
  3 Ірина Бондар             PI-22     95.20
  9 Катерина Лисенко         PI-23     93.10
  1 Анна Коваль              PI-21     91.50
  4 Андрій Шевченко          PI-21     88.70
  2 Олег Мельник             PI-21     84.00
  5 Марія Бойко              PI-22     79.30
  6 Тарас Гриценко           PI-23     72.40
 10 Максим Мельник           PI-21     70.60
  7 Софія Гнатюк             PI-22     66.80
  8 Дмитро Ткаченко          PI-23     58.90

Сортування за (group, average_grade) у зворотному порядку: [9, 6, 8, 3, 5, 7, 1, 4, 2, 10]
Pipeline: групи PI-21 і PI-22 -> сортування -> топ-3: [3, 1, 4]

*args: calculate_stats(75, 80, 92, 96) ->
  GradeStats(size=4, average=85.75, minimum=75, maximum=96)
**kwargs: create_student(id=11, first_name=..., ..., average_grade=89.0) ->
  {'id': 11, 'first_name': 'Тарас', 'last_name': 'Новак', 'group': 'PI-23', 'average_grade':
89.0}
**kwargs з балом 120 -> ValueError: Середній бал має бути в межах 0-100

Підсумок (build_summary(**options)):
  statistics: {'size': 10, 'average': 80.05, 'minimum': 58.9, 'maximum': 95.2}
  best_student: Ірина Бондар
  students_by_group: {'PI-21': 4, 'PI-22': 3, 'PI-23': 3}
  top_students: [3, 9, 1]
  min_grade: 85
  students_above_min: 4
Експортовано у results\summary.json

Останні операції (deque, maxlen=5):
  ['фільтр за мін. балом', 'сортування за балом', 'сортування за балом', 'сортування за балом',
'фільтр за мін. балом']
Усього викликів (Counter): {'пошук за ID': 2, 'групування за групою': 2, 'фільтр за мін.
балом': 2, 'сортування за балом': 3}
```

Ручна перевірка: сума балів за групами 334.8 + 241.3 + 224.4 = 800.5, середній бал 800.5 / 10 = 80.05, що збігається з виводом програми; найвищий бал у списку – 95.2 (Ірина Бондар).

8. Аналіз складності операцій

Таблиця 6 – Часова складність операцій програми (n – кількість студентів)

Операція	**Реалізація**	**Складність**
побудова dict-індексу	`create_student_index`	O(n)
пошук за ID у list	`find_student_linear`	O(n), у середньому n/2 порівнянь
пошук за ID в індексі	`find_student` (dict.get)	O(1) у середньому
перевірка `x in set`	`'PI-24' in groups`	O(1) у середньому
фільтрація	`filter_by_min_grade`	O(n)
групування / підрахунок	`group_students`, `count_students_by_group`	O(n)
вкладене групування	`group_nested` (k рівнів)	O(n·k)
середнє, максимум	`calculate_average`, `find_best_student`	O(n), пам'ять O(1)
сортування	`sort_by_grade`, `sort_items`	O(n log n) (Timsort)
топ-k студентів	`sort_by_grade(...)[:k]`	O(n log n); heapq.nlargest – O(n log k)

Операція	Реалізація	Складність
запис у журнал	`deque.append` з maxlen	O(1)

Якщо пошуків за ID багато, вигідно один раз побудувати індекс за O(n) і далі шукати за O(1) замість O(n) на кожен пошук (перевірено в розділі 9).

9. Експериментальна частина (benchmark)

Для 1 000, 10 000 і 100 000 згенерованих студентів виміряно середній час пошуку 200 випадкових ID (`timeit`, найкращий із 5 повторів): A – лінійний пошук у list, B – `index.get` у dict-індексі, а також `x in list`, `x in set` і час побудови індексу (`python -m data_processor.benchmark`).

Таблиця 7 – Результати benchmark (Python 3.12, Windows 11)

Кількість records	list search, мкс	dict search, мкс	dict швидше, разів	x in list, мкс	x in set, мкс	побудова індексу, мс
1 000	8.14	0.019	436	1.57	0.023	0.024
10 000	84.41	0.018	4 687	16.33	0.022	0.436
100 000	830.94	0.019	43 638	173.19	0.020	5.188

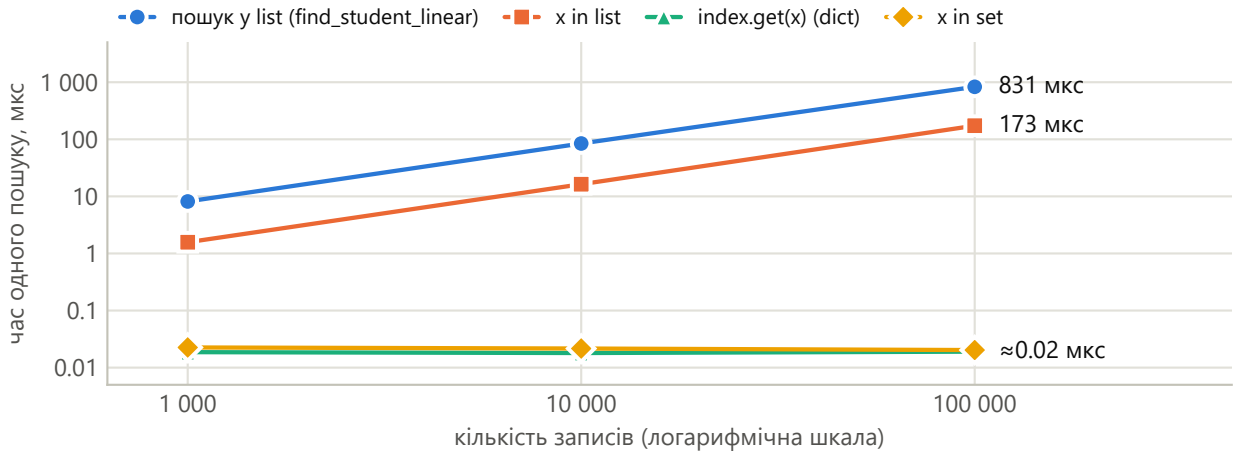


Рисунок 1 – Час одного пошуку залежно від кількості записів (обидві осі логарифмічні)

При збільшенні кількості записів у 10 разів час пошуку в list зростає в 10.4 та 9.8 раза – лінійна залежність O(n), на графіку це пряма з нахилом 1. Час пошуку в dict майже не змінюється (≈0.02 мкс, розкид 1.1×) – це O(1). На 100 000 записів індекс швидший у 43 638 разів. `x in list` приблизно в 4.8 раза швидший за `find_student_linear`, бо цикл виконується в C, але залишається O(n); `x in set` працює так само швидко, як dict (на графіку їхні лінії збігаються).

Побудова індексу коштує O(n) (5.2 мс на 100 000 записів) і окупається вже після 3–6 пошуків. Отже, теоретична складність добре узгоджується з практичним часом: для багаторазового пошуку за ключем потрібно використовувати dict або set.

10. Перевірка правильності

Тести (`tests/`, unittest) перевіряють пошук обома способами, межі фільтрів, closure, групування, сортування, pipeline, валідацію, агрегації, експорт JSON, декоратори і benchmark. Пакет встановлено в чисте venv через `pip install -e .`, console commands працюють.

Таблиця 8 – Результати перевірок

Перевірка	**Результат**
`python -m unittest discover -s tests`	Ran 33 tests in 8.812s — OK
`ruff check src tests`	All checks passed!
`ruff format --check src tests`	9 files already formatted
`mypy src tests`	Success: no issues found in 9 source files

Усі обов'язкові вимоги розділу 25 методички виконано (розділи 1–9 звіту). Із завдань підвищеної складності реалізовано всі десять: універсальні фільтрація і сортування, параметризований декоратор, closures для фільтрів, вкладене групування, Counter, deque-журнал, benchmark list vs set/dict, експорт у JSON і pipeline.

Висновки

Створено модуль аналізу успішності студентів (варіант 1), що продовжує ЛР1: пошук, фільтрація, групування, сортування, агрегація і статистика. Структури обрано за властивостями: list – впорядкований набір, tuple – незмінні константи і складені ключі, set – унікальність і швидка перевірка належності, dict – записи та індекс, collections – групування, підрахунок і журнал операцій. Функції вищого порядку, closures і decorators дозволили змінювати поведінку без зміни основного коду. Експеримент підтвердив теорію: час пошуку в list зростає лінійно, у dict – сталий, і на 100 000 записів різниця перевищує 43 000 разів.